

Set Environment Variables in Postman

Capturing GoRest user id into a Global Variable using a Post-Response Script

Hands-On	12
Scenario	Set the environment variables in Postman
Tool	Postman + GoRest API (https://gorest.co.in)
Method	POST
Endpoint URL	https://gorest.co.in/public/v2/users
Auth	Bearer Token (from GoRest profile)
Outcome	Response id captured into global variable APIID
Duration	10 minutes
Prerequisite	Workspace and Collection already created

Objective

The objective of this assignment is to send a **POST** request to the GoRest **Create User** API and use a **post-response script** to capture the newly-created user's **id** into a Postman **Global Variable** named **APIID**. Once captured, the id can be referenced from any other request in the workspace using `{{APIID}}` — this is the foundation of request chaining (e.g. fetching, updating, or deleting the user just created).

About GoRest

GoRest is a free public sandbox REST API designed for testing and prototyping. Read endpoints (GET) are open to everyone, but write endpoints (POST, PUT, PATCH, DELETE) require a personal access token issued from the user's GoRest account. Each token is rate-limited and user-scoped — data created with one token is only visible to that user, and the entire dataset resets daily.

Note — Email Must Be Unique

GoRest enforces a unique-email constraint on the Create User endpoint. If the same email address is sent twice, the second call returns **422 Unprocessable Entity** with a validation error. To avoid this, use a fresh email address each time — for example by appending a timestamp:
`tenali_{{timestamp}}@example.com`. Postman replaces `{{timestamp}}` with the current Unix epoch on every send.

Steps

1 Go to the Right Workspace

Open Postman and sign in if not already logged in.

Click the **Workspaces** dropdown in the upper-left corner and select the workspace used in earlier assignments.

2 Register / Login on GoRest and Generate a Token

Open <https://gorest.co.in/> in a browser.

Click **Sign Up** (or **Login**) and authenticate using a personal email account — GoRest supports Google, GitHub and email/password sign-in.

Once logged in, click the user **Profile** icon in the top-right corner and choose **Access Tokens** (or *API Token*).

Click **Create New Token**, then click **Copy** to copy the generated token to the clipboard. Save it somewhere safe.

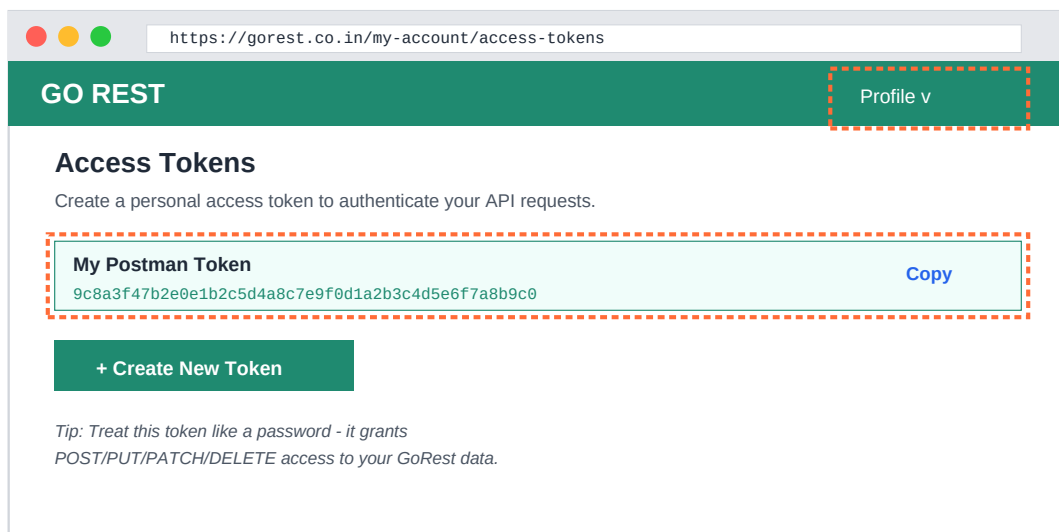


Figure 1 — GoRest profile page showing the generated access token. Use the Copy button to capture it for use in Postman.

3 Create a New Request in Postman

Inside the right collection, click the **+** button or **Add a request** link to create a new request.

Rename it to something descriptive such as *Create User* (or *POST GoRest Create User*).

4 Set Method and URL

Set the HTTP method to **POST**.

Enter the URL:

<https://gorest.co.in/public/v2/users>

5 Configure Authorization (Bearer Token)

Click the **Authorization** tab below the URL bar.

From the **Type** dropdown, choose **Bearer Token**.

Paste the GoRest token (from Step 2) into the **Token** field on the right.

Postman now adds an **Authorization: Bearer <token>** header to every send of this request.

6 Add the Request Body (Sample JSON)

Click the **Body** tab.

Choose the **raw** radio option and select **JSON** from the format dropdown on the right.

Paste the JSON payload shown in the **Sample Request Body** section below into the editor.

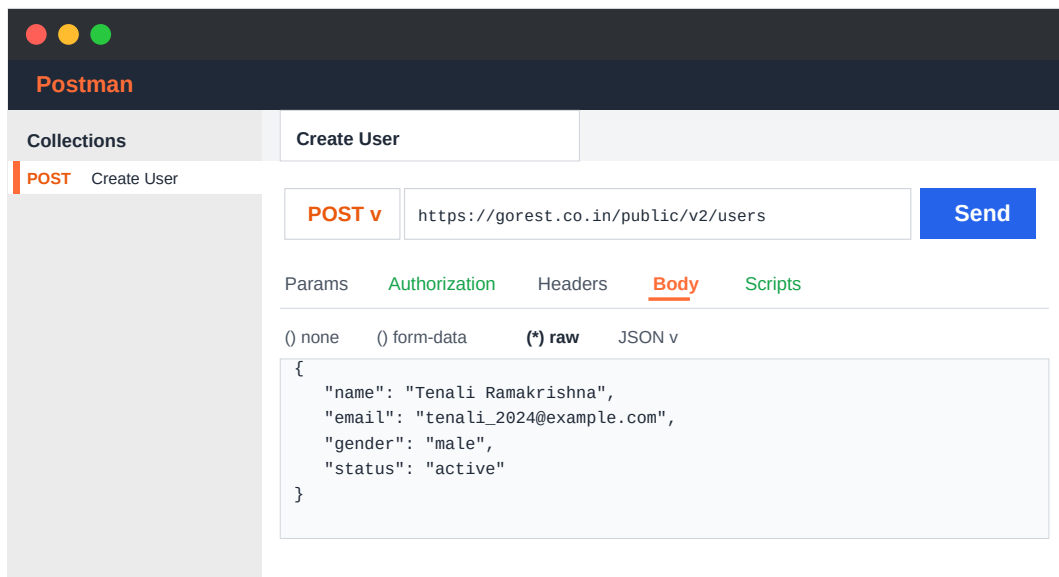


Figure 2 — Postman request fully configured: **POST** method, GoRest URL, and the JSON request body with the four required fields.

Sample Request Body (Input)

GoRest's Create User endpoint requires four fields: **name**, **email**, **gender**, and **status**. The email must be unique across all users:

```
{
  "name"    : "Tenali Ramakrishna",
  "email"   : "tenali_{{timestamp}}@example.com",
  "gender"  : "male",
  "status"  : "active"
}
```

7

Add the Post-Response Script to Capture id into APIID

Click the **Scripts** tab (in older Postman versions this is labelled **Tests**).

Switch to the **Post-response** sub-tab — this is where JavaScript runs *after* the response arrives.

Paste the JavaScript shown in the **Post-Response Script** section below.

The first two lines parse the response JSON and save the id field into a global variable called APIID. The block at the bottom asserts the status code is 201.

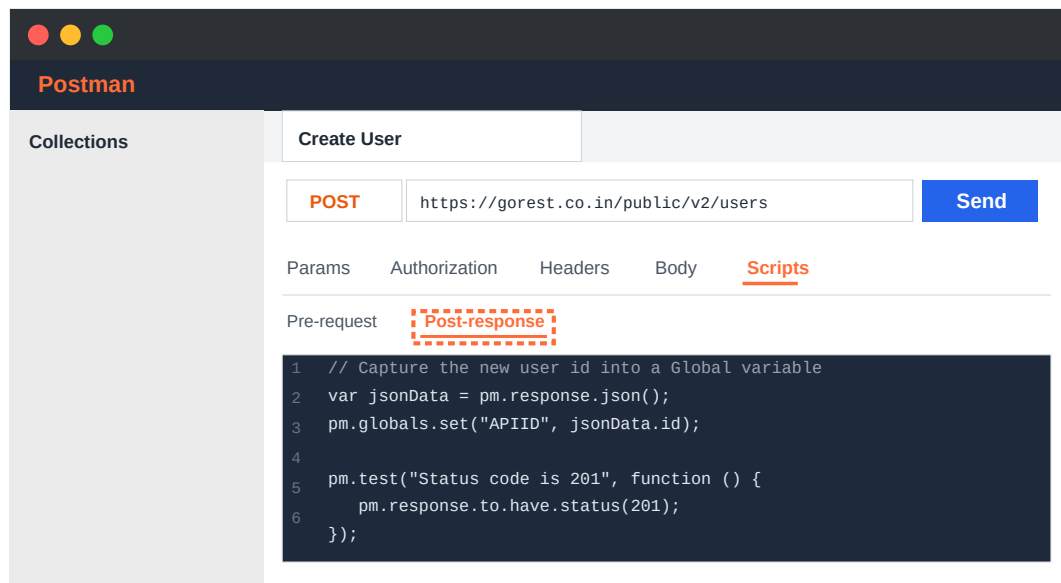


Figure 3 — **Scripts** tab in **Post-response** mode with the JavaScript that captures the user id into a Postman global variable named **APIID**.

Post-Response Script (Input)

Paste this JavaScript into the **Post-response** editor. It captures the new user's id into a global variable and asserts the status code:

```
// Capture the new user id into a Global variable
var jsonData = pm.response.json();
pm.globals.set("APIID", jsonData.id);

pm.test("Status code is 201", function () {
    pm.response.to.have.status(201);
});
```

8

Click Send and Verify the Response

Click the blue **Send** button.

Verify the following on the response:

- **Status** — should be **201 Created** in green.
- **Body** — contains a JSON object with an **id** field (a positive integer) plus the name, email, gender, and status echoed back.
- **Test Results** tab — should show **1 / 1 passed** for “Status code is 201”.

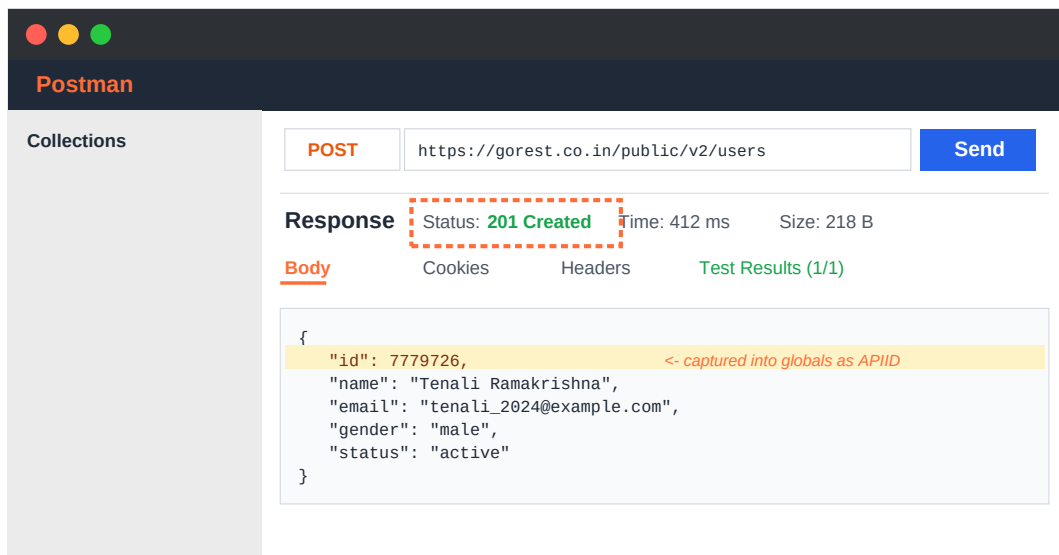


Figure 4 — Response panel with **201 Created**, the `id` field highlighted (it has been captured into the `APIID` global variable), and other user fields echoed back.

Verify the Global Variable Was Set

To confirm `APIID` was actually written:

- 1 In the top-right of the Postman workbench, click the **Eye** (Environment Quick Look) icon. Scroll to the **Globals** section — `APIID` should be listed with the captured user id.
- 2 Or open **Environments** → **Globals** from the left sidebar — the variable shows up in the variables table.
- 3 It can now be referenced in any subsequent request as `{{APIID}}`, e.g. in a follow-up GET `https://gorest.co.in/public/v2/users/{{APIID}}` to fetch the same user back.

Expected Response

On success, GoRest returns **HTTP 201 Created** with the new user record. The exact id value will differ on each call:

```
{
  "id": 7779726,
  "name": "Tenali Ramakrishna",
  "email": "tenali_1714765432@example.com",
  "gender": "male",
  "status": "active"
}
```

Verification Checklist

- 1 The response status is **201 Created**.
- 2 The response body contains all four fields plus a generated **id**.
- 3 The **Test Results** tab shows **1/1 passed** for the “Status code is 201” assertion.
- 4 The Globals section (Environment Quick Look) lists `APIID` with the captured user id.
- 5 Request is saved (**Ctrl + S** / **Cmd + S**) inside the collection.

Conclusion

A POST request was sent to GoRest's Create User endpoint, the response verified, and the user id captured into a global variable named APIID using a post-response script. This is one of the most useful patterns in Postman — capturing a value from one response and feeding it into subsequent requests. The same technique applies to authentication tokens, resource ids, and any other dynamic data that needs to flow between requests in a collection or run.